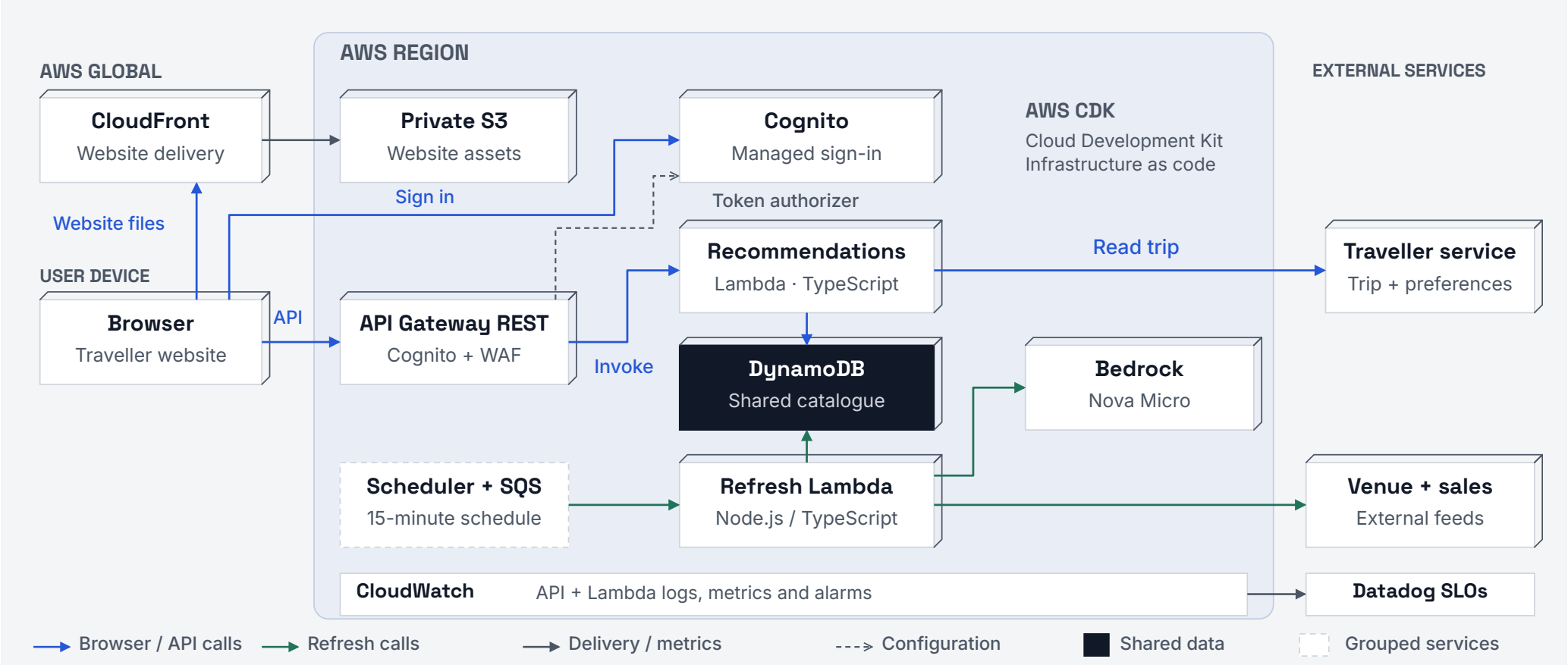
Airport Trip Planner: solution design

The planner recommends airport restaurants and shops using a traveller's preferences, flight time and current sales. A pre-built AI model interprets shop descriptions in the background when venue data is updated. Requests use the prepared data, so recommendations remain fast when the model is unavailable. Results depend on accurate venue information and estimated visit times.

1. AWS infrastructure



Technology choices and trade-offs

Private S3 + CloudFront
Static hosting without servers. **The backend runs in one region, so a regional outage interrupts recommendations.**

API Gateway REST, Cognito + WAF
Handles sign-in and protects API access. **Assumes the traveller service accepts Cognito access tokens.**

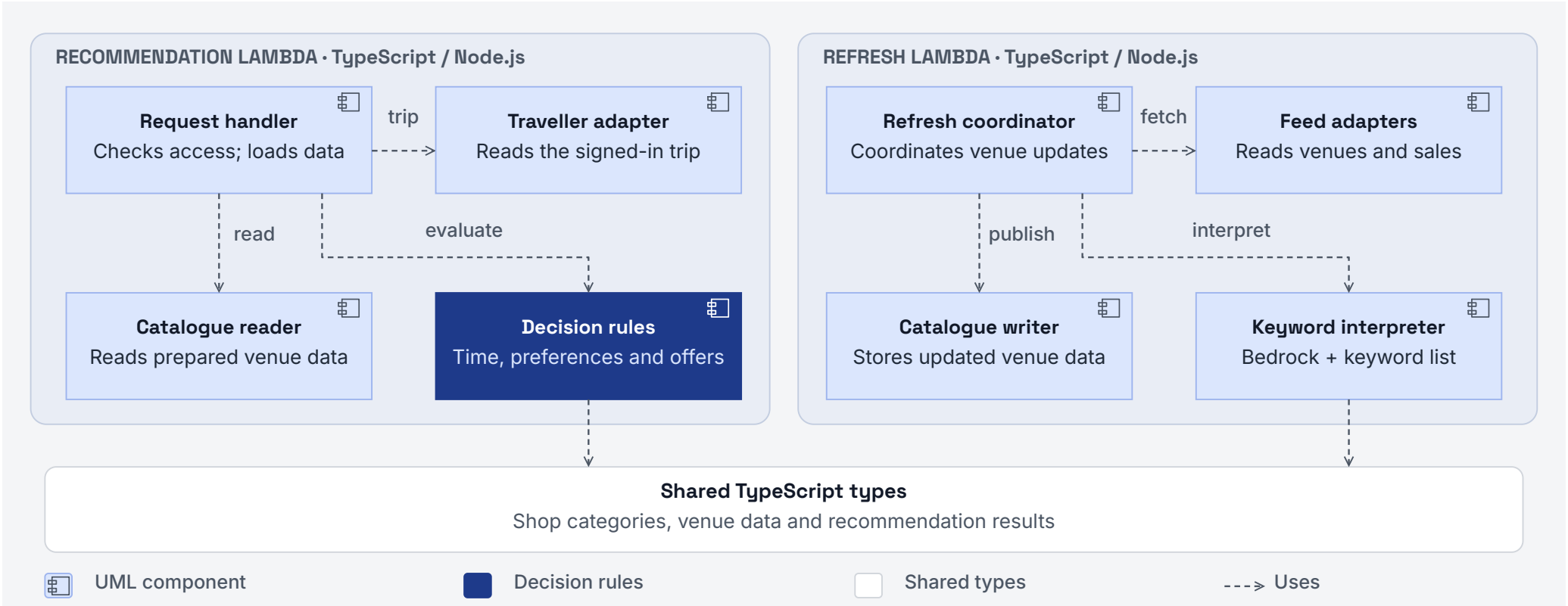
Lambda + DynamoDB
Functions scale separately. Recommendations read a complete version of the venue data. **Starting a new function instance adds delay.**

EventBridge Scheduler + SQS
Schedules venue updates and buffers them when they take longer. **Venue changes appear after the next successful update.**

Bedrock (Nova Micro)
A pre-built AI model interprets the different words brands use to describe their shops. **This avoids training a model, but adds running costs.**

CloudWatch + Datadog
CloudWatch collects logs and metrics; Datadog tracks targets. **Using Datadog adds cost and shares monitoring data with an external provider.**

2. Backend components



Recommendation rules

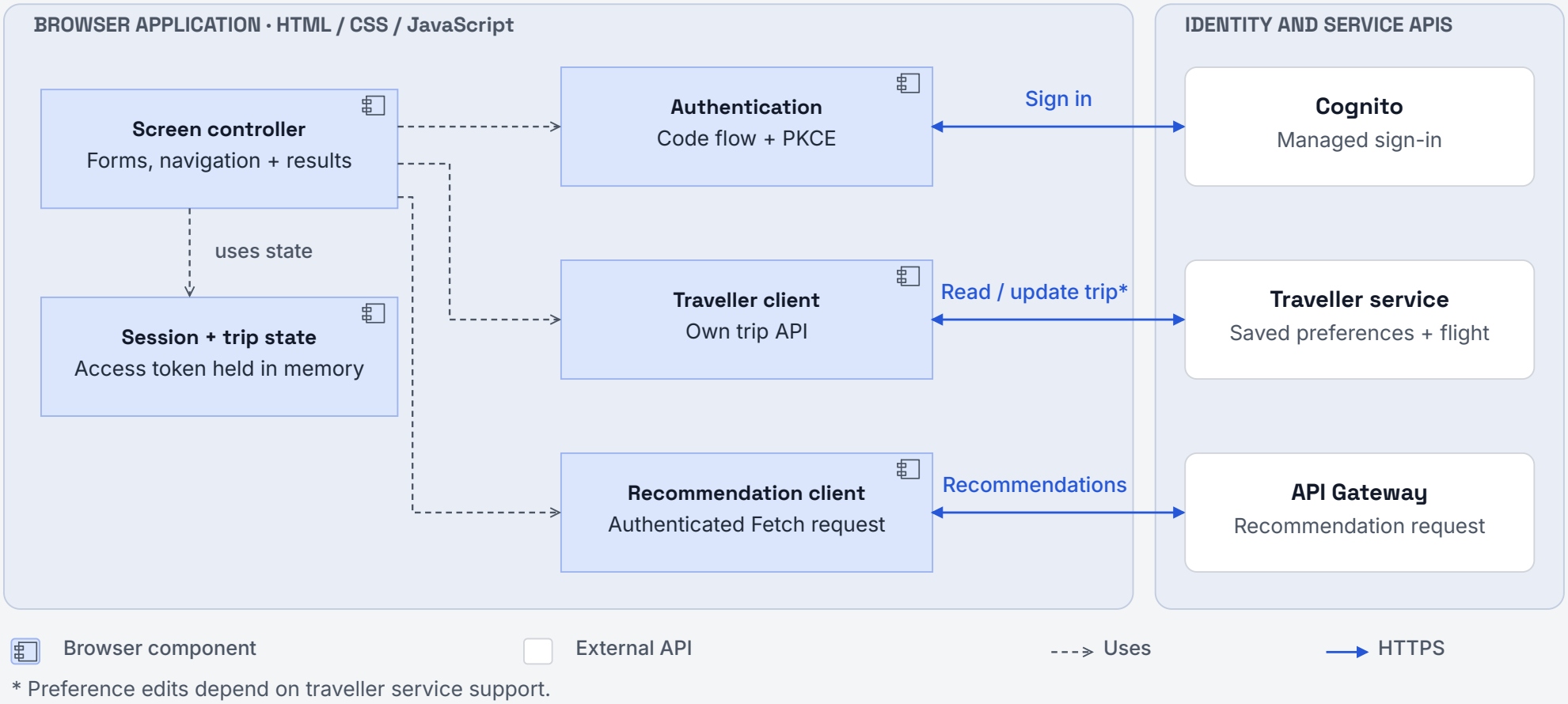
Input	Effect on recommendations
Flight time	Recommend places the traveller has time to visit before boarding. With too little time, advise going to the gate.
Preferences	Show matching places first. Dietary requirements use information supplied by the venue.
Sales	Promote valid offers among suitable places. Without preferences, places with offers appear first. Sales data comes from an assumed external feed.
Brand descriptions	Recognise that different words can mean the same thing. For example, “cheap” and “budget” both describe an affordable option.

Technology and quality objectives

TypeScript / Node.js provides shared data types for both functions.

Quality goal	Measure and 30-day design target
Availability	At least 99.9% of valid, authorised requests return recommendations, gate advice or an explanation that no suitable venue is available. Failures in the planner or its dependencies count as unavailable.
Response time	At least 95% of the same requests return a useful result within two seconds, measured at API Gateway. Delays in the traveller service affect this target.
Current venue information	At least 99% of requests that check venue data use information updated within the last 30 minutes.

3. Frontend design



The browser reads the trip to display it. The backend gets the trip separately to calculate recommendations.

Browser technology choices

Technology	Reason and trade-off
HTML / CSS / JavaScript	A small, static website with no framework to maintain. **A framework would make a larger website easier to organise and maintain.**
Cognito + PKCE	Handles sign-in; the APIs limit each traveller's access to their own trip. **Tokens stay in memory, so reloading restarts the browser session.**
DOM + semantic HTML	Shop descriptions are displayed as text to prevent embedded code from running. Labelled controls, keyboard navigation and clear status messages support accessibility.